GUÍA DE LABORATORIO

CVA6 en KR260

De la conexión a la evidencia.

Arranque · XSDB · Vivado · ILA · Timing

90,9 MHz

IMPLEMENTACIÓN CERRADA

25 MHz

REFERENCIA FUNCIONAL

ANTES DE LA PRUEBA

90,9 MHz: validación en placa pendiente.

Procedimiento manual y automatizado

Comandos completos, diagnóstico y preguntas para la reunión.

Edición local / 30 octubre 2026

Encontrar cada paso

Índice interactivo. Hacé clic en un apartado para ir a su página.

Cómo utilizar este manual	03
Paso 01 / conectar y encender	04
Paso 02 / verificar los archivos	05
Paso 03 / abrir herramientas y registrar	06
Paso 04 / elegir perfil y conectar XSDB	07
Paso 05 / reset y primera ejecución FSBL	08
Paso 06 / programar PL y repetir FSBL	09
Paso 07 / comprobar estado antes del arranque	10
Paso 08 / asociar probes y armar ILA	11
Paso 09 / lanzar aplicación y leer estados	12
Paso 10 / decidir PASS, FAIL o incompleto	13
Paso 11 / descargar y conservar captura	14
Paso 12 / cerrar y preparar la siguiente pasada	15
A / Alternativa automatizada	16
B / Consolas y herramientas	17
C / Programar el BIT desde la UI	18
D / Diagnóstico y recuperación	19
E / Consultar timing en Vivado	20
F / Profesor: resultados	21
G / Profesor: arquitectura y prueba	22
H / Profesor: timing y evidencia	23
I / Rutas: proyecto y placa	24
J / Rutas: reportes y documentación	25
K / Acceso rápido y referencias	26

01 Cómo utilizar este manual

Objetivo: ejecutar la prueba existente de BootROM/DDR del CVA6 en KR260, primero a25 MHz como control y después a90,908180 MHz. No se modifica el CPU ni la ROM.

Leer este apartado y seguir los pasos 01 a 12 EN ORDEN. No hace falta recorrer los apéndices para realizar la vía principal. Cada paso indica la consola, la acción, la salida esperada y cuándo detenerse.

El procedimiento principal es manual: terminal Linux + XSDB + Vivado. Si se prefiere la rutina automatizada, ir al apéndice A y no realizar ambos procedimientos simultáneamente.

Primera pasada: seleccionar frecuencia 25 en XSDB y en Vivado. Completar los pasos y guardar evidencia. Segunda pasada: comenzar desde una sesión nueva, seleccionar 91 en ambas herramientas y repetir. Nunca cambiar sólo el divisor o mezclar BIT/LTX/FSBL.

25 significa24,999750MHz;91 significa90,908180 MHz. El paquete91 tiene STA cerrado (WNS+0,001ns;TNS0;WHS+0,004ns), pero su funcionamiento hardware sigue pendiente al preparar esta edición.

Qué se considera éxito: estado de BootROM PASS, estados secundarios sin diferencias, reset liberado y captura ILA con muestras si se hizo captura. No es validación exhaustiva de ISA, Linux o memoria.

La aplicación ARM escribe patrones en DDR antes de liberar CVA6. No preserva programas precargados. No se toca QSPI ni se genera BOOT.BIN con esta guía.

Organización: procedimiento principal; alternativas y diagnóstico; preguntas para el profesor; rutas y referencias. Si una orden falla, detenerse en ese punto, guardar el mensaje y consultar diagnóstico. No continuar pegando el bloque siguiente.

02 Paso 01 / conectar y encender

DÓNDE: mesa de trabajo y terminal Linux. ESTADO INICIAL: placa sin alimentar.

1. Identificar que la placa sea KR260. Colocarla estable y mantener libre su ventilación. Tener a mano su fuente y un cable micro-USB de DATOS. La notebook debe tener cargador conectado.

2. Con la placa sin alimentar, identificar J4: micro-USB UART/JTAG. No confundirlo con los conectores USB host para periféricos. Conectar ahí el extremo micro-B del cable; conectar el otro extremo a la notebook.

3. Identificar J12, entrada de alimentación. Conectar la fuente adecuada de la placa y alimentarla. Comprobar los indicadores de alimentación. Si no hay indicación de encendido, no intentar solucionar por software.

4. En esta prueba JTAG no hace falta preparar una microSD, monitor, teclado ni Ethernet. No estamos ejecutando el tutorial de Linux ni arrancando un sistema operativo en CVA6.

5. No pulsar SW1/FWUEN: es el botón de actualización/recuperación de firmware, no el control del clock ni el selector habitual de esta prueba. No modificar jumpers o resistencias para esta guía. El script cambia el modo de arranque de forma volátil por registros.

6. Abrir una terminal y comprobar USB con los comandos de abajo. Registrar cualquier error. Que aparezca el puente USB no prueba todavía que JTAG pueda abrirse.

7. Si la placa estaba ejecutando Linux u otra aplicación, el reset de sistema de este procedimiento la interrumpe. No ejecutar dos pruebas simultáneas.

Fuente de identificación física: documentación AMD KR260, conexión J4/J12 y UG1092 Firmware Update; enlaces completos en el apéndice.

TERMINAL / Bash

```
lsusb
pgrep -af hw_server
# Espacio disponible y hora para identificar evidencia:
df -h /home/nachofalchi/data/fpga-work
date -Is
```

Si USB no aparece: revisar alimentación, otro puerto de la notebook y un cable de datos conocido. No asumir que encender LEDs significa tener un cable USB de datos. Si hay problemas de permisos, guardar el error exacto; no ejecutar Vivado como root ni instalar drivers sin diagnosticar.

COMPROBACIÓN: alimentación presente y puente USB visible en lsusb. DETENERSE si la placa no enciende o USB no aparece. SIGUIENTE: paso02.

03 Paso 02 / verificar los archivos

DÓNDE: terminal Linux, antes de tocar registros del hardware.

ACCIÓN: comprobar los paquetes sin depender del wrapper cva6. Copiar el bloque completo. Se calculan hashes; no se programa la FPGA.

El paquete25 contiene un error histórico únicamente en la autoentrada de su manifiesto. Por eso se filtra ESA entrada y se verifican todos los archivos de contenido. No se omite ninguna entrada ELF, BIT o LTX. El paquete91 se verifica completo.

TERMINAL / Bash

```
CVA6_BASE=/home/nachofalchi/data/fpga-work
CVA6_BOOT="$CVA6_BASE/cva6-equipo/corev_apu/fpga/software"
CVA6_BOOT="$CVA6_BOOT/kr260_ps_boot"
CVA6_KG="$CVA6_BOOT/known_good_2026-09-18"
CVA6_PK="$CVA6_BOOT/timing_closed_91mhz_2026-10-22"
(cd "$CVA6_KG" && awk '$2 !~ /SHA256SUMS.txt$/ {print}' \
  SHA256SUMS.txt | sha256sum -c -)
(cd "$CVA6_PK" && sha256sum -c SHA256SUMS.txt)
CVA6_FSBL="$CVA6_BASE/operacion/cva6_ops/fsbl91"
sha256sum "$CVA6_FSBL/build_20261029_1831/kr260_fsbl_91mhz_patched.elf"
```

ESPERADO: 17 archivos de contenido de 25 MHz y 29 archivos de 91 correctos correctos. SHA256 del FSBL91 (unir mitades):

4b2c98d7ea93099d62b7538ce4d92b071

d3672ef157b48acf7bf688df380ab87

DETENERSE ante archivo faltante o hash diferente. SIGUIENTE: paso03.

04 Paso 03 / abrir herramientas y registrar

DÓNDE: terminal Linux. Dejar una terminal para XSDB y una ventana de Vivado. La grabación es opcional pero recomendable: primero crea una carpeta nueva y luego inicia una subshell grabada.

1. Ejecutar el bloque. Si script no existe, guardar manualmente el texto de XSDB al final; no impide ejecutar la prueba.

2. Dentro de la subshell que abre script, iniciar XSDB con la última orden.

3. Si Vivado ya está abierto, reutilizarlo. Si no, iniciarlo desde otra terminal con la ruta que figura abajo.

4. Para el servidor, usar Hardware Manager > Open Target > Auto Connect o iniciar hw_server si todavía no hay uno activo. No iniciar dos servidores sobre 3121.

TERMINAL / Bash

```
CVA6_BASE=/home/nachofalchi/data/fpga-work
CVA6_RUN="$CVA6_BASE/operacion/cva6_ops/manual_$(date +%Y%m%d_%H%M%S)"
mkdir -p "$CVA6_RUN"
echo "$CVA6_RUN"
script -q -f "$CVA6_RUN/xsdb_terminal.log"
# En la subshell grabada:
/home/nachofalchi/data/fpga-work/Xilinx/2026.1/\
Vitis/bin/xsdb
```

ESPERADO: prompt xsdb%. La carpeta impresa es la de evidencia, anotarla.

Vivado: /home/nachofalchi/data/fpga-work/Xilinx/2026.1/Vivado/bin/vivado

Servidor opcional: misma ruta Vivado/bin/hw_server. Mantener su terminal abierta.

En Vivado: Flow > Open Hardware Manager > Open Target > Auto Connect. Todavía no pulsar Program Device. Si falla, revisar cable y servidor. SIGUIENTE: paso04.

05 Paso 04 / elegir perfil y conectar XSDB

DÓNDE: XSDB, prompt xsdb%. Mantener esta sesión abierta hasta el paso12.

ACCIÓN: en la primera pasada dejar set frecuencia 25. En la segunda cambiar únicamente esa selección a 91 ANTES de ejecutar el bloque completo. Las rutas se derivan del perfil; no editarlas a mano.

Conectar una sola placa. sel exige un único target exacto e imprime sus propiedades; confirmar que PSU y A53 pertenecen al mismo cable. El selector manual no reemplaza todos los controles de identidad de la automatización.

XSDB / Tcl

```
set base /home/nachofalchi/data/fpga-work
set boot [file join $base cva6-equipo corev_apu fpga \
    software kr260_ps_boot]
# PRIMERA pasada: 25. SEGUNDA pasada nueva: 91.
set frecuencia 25
if {$frecuencia == 25} {
    set pkg [file join $boot known_good_2026-09-18]
    set fsbl [file join $pkg kr260_fsbl_patched.elf]
    set clock_expected 0x01012800
} elseif {$frecuencia == 91} {
    set pkg [file join $boot timing_closed_91mhz_2026-10-22]
    set fsbl [file join $base operacion cva6_ops fsbl91 \
        build_20261029_1831 kr260_fsbl_91mhz_patched.elf]
    set clock_expected 0x01010b00
} else {error "Frecuencia no admitida"}
set bit [file join $pkg kr260_top.bit]
set app [file join $boot known_good_2026-09-18 kr260_app_jtag.elf]
puts "PERFIL=$frecuencia; BIT=$bit; FSBL=$fsbl"
foreach p [list $bit $fsbl $app] {
    if {![file isfile $p]} {error "Falta archivo: $p"}
}
proc sel {name} {
    set f [format {name == "%s"} $name]
    set ts [targets -target-properties -filter $f]
    if {[llength $ts] != 1} {error "Target ambiguo: $name"}
    set t [lindex $ts 0]
    puts $t
    targets [dict get $t target_id]
}
connect -url tcp:127.0.0.1:3121
after 4000
targets
sel PSU
```

ESPERADO: sin errores de archivos, lista de targets y PSU seleccionado. Anotar el cable serial. DETENERSE si faltan targets o aparece ambigüedad. No seleccionar otro procesador por tanteo.

Las variables de XSDB NO existen en Vivado. Allí se seleccionará el mismo perfil otra vez. SIGUIENTE: paso05.

06 Paso 05 / reset y primera ejecución FSBL

DÓNDE: misma sesión XSDB del paso04. ESTADO INICIAL: perfil elegido y PSU accesible.

ACCIÓN: cambiar el arranque a JTAG volátil, resetear el sistema y ejecutar el FSBL seleccionado en Cortex-A53 #0. La espera de 15 segundos no indica PASS por sí sola.

Qué hace cada orden: mwr escribe registros de control; rst -system reinicia el sistema; dow carga el ELF; con deja correr ARM; stop lo detiene para continuar. after usa milisegundos.

XSDB / Tcl

```
# Seleccionar arranque JTAG volátil y resetear sistema.
sel PSU
mwr 0xffca0010 0x0
mwr 0xff5e0200 0x0100
rst -system
after 4000

# Primera ejecución de FSBL.
sel {Cortex-A53 #0}
rst -processor
after 1500
dow -force $fsbl
con
after 15000
stop
```

ESPERADO: descarga del FSBL y parada del ARM sin errores. Todavía no se ejecutó la prueba de CVA6.

DETENERSE si rst, dow, con o stop fallan. Guardar el error y revisar conectividad/ELF. SIGUIENTE: paso06.

07 Paso 06 / programar PL y repetir FSBL

DÓNDE: misma sesión XSDB. ESTADO INICIAL: primera ejecución de FSBL terminada.

ACCIÓN: cargar el BIT del perfil y ejecutar OTRA VEZ el FSBL. El segundo pase es necesario en esta receta para restablecer correctamente la inicialización tras programar PL.

El reset final sólo afecta al A53 seleccionado. CVA6 debe seguir retenido por el FSBL parcheado. No cargar todavía la aplicación y no cambiar VIO.

XSDB / Tcl

```
# Programar el BIT cerrado (no el de impl_1).
sel PSU
fpga -f $bit
after 3000

# Segunda ejecución de FSBL tras configurar PL.
sel {Cortex-A53 #0}
rst -processor
after 1500
dow -force $fsbl
con
after 15000
stop
rst -processor
after 1500
```

ESPERADO: configuración PL sin errores, segunda descarga FSBL finalizada y A53 detenido. No hace falta usar Program Device: fpga -f ya hizo ese trabajo.

DETENERSE ante error de programación o ejecución. SIGUIENTE: paso07.

08 Paso 07 / comprobar estado antes del arranque

DÓNDE: XSDB, con las variables del perfil todavía definidas.

ACCIÓN: ejecutar el bloque completo. rd32 lee una palabra; rd64 reconstruye un valor de 64 bits a partir de dos palabras de 32 bits. Se exige clock del perfil, reset retenido, PL configurada, DDR normal y HP0 de 64 bits.

XSDB / Tcl

```
proc rd32 {a} {
    return [expr {[lindex [mrd -value $a 1] 0] & 0xffffffff}]
}
proc rd64 {a} {
    set w [mrd -value $a 2]
    if {[llength $w] != 2} {error "Lectura incompleta"}
    return [expr {([lindex $w 0] & 0xffffffff) | \
        (([lindex $w 1] & 0xffffffff) << 32)}]
}
set pl0 [rd32 0xff5e00c0]
puts [format "PL0_REF_CTRL=0x%08x" $pl0]
if {($pl0 & 0x013f3f07) != $clock_expected} {
    error "Clock distinto del perfil elegido"
}
set g [rd32 0xff0a0054]
set d [rd32 0xff0a0344]
set o [rd32 0xff0a0348]
if {($g & 0x80000000) != 0 || \
    ($d & 0x80000000) == 0 || ($o & 0x80000000) == 0} {
    error "CVA6 no esta retenido en reset"
}
if {([rd32 0xffca3010] & 8) == 0} {
    error "PL sin configurar"
}
if {([rd32 0xfd070004] & 7) != 1} {
    error "DDR fuera de modo normal"
}
foreach a {0xfd380000 0xfd380014} {
    if {([rd32 $a] & 3) != 1} {error "HP0 no es 64 bits"}
}
puts "PREPARACION OK; aun no es PASS funcional"
```

ESPERADO: PREPARACION OK; aun no es PASS funcional. DETENERSE si cualquiera de los controles da error. No modificar un registro aislado para saltar la comprobación.

SIGUIENTE: paso08. Desde ahora hasta lanzar la app, no resetear PS/ARM ni volver a programar PL.

09 Paso 08 / asociar probes y armar ILA

DÓNDE: Vivado, Hardware Manager. XSDB queda abierto y detenido en el estado preparado.

1. Abrir Flow > Open Hardware Manager y Open Target > Auto Connect si no estaba conectado. Confirmar la misma placa/cable que en XSDB. No pulsar Program Device.

2. En Tcl Console de VIVADO, ejecutar el bloque siguiente. Elegir el mismo25 o 91 que en XSDB. El bloque selecciona un único dispositivo e ILA, carga el LTX y lo arma para commit_valid=1.

3. Esperar a que la UI indique espera de trigger. No ejecutar todavía wait_on_hw_ila: primero hay que arrancar la app en XSDB.

La captura ILA es parte de la ruta principal. Si se omite intencionalmente, registrar que el resultado funcional no incluye captura y saltar del paso07 al09.

VIVADO / Tcl Console

```
set base /home/nachofalchi/data/fpga-work
# Debe ser el MISMO perfil elegido en XSDB.
set frecuencia 25
set boot [file join $base cva6-equipo corev_apu fpga \
    software kr260_ps_boot]
if {$frecuencia == 25} {
    set pkg [file join $boot known_good_2026-09-18]
} elseif {$frecuencia == 91} {
    set pkg [file join $boot timing_closed_91mhz_2026-10-22]
} else {error "Frecuencia no admitida"}
set ds [get_hw_devices]
if {[llength $ds] != 1} {error "Dispositivo ambiguo"}
set dev [lindex $ds 0]
current_hw_device $dev
set_property PROBES.FILE [file join $pkg kr260_top.ltx] $dev
refresh_hw_device $dev
set cores [get_hw_ilas -of_objects $dev]
if {[llength $cores] != 1} {error "ILA ausente/ambiguo"}
set ila [lindex $cores 0]
set p [get_hw_probes m_debug_commit_valid -of_objects $ila]
if {[llength $p] != 1} {error "Probe ausente/ambiguo"}
reset_hw_ila $ila
set_property CONTROL.TRIGGER_MODE BASIC_ONLY $ila
set_property CONTROL.TRIGGER_CONDITION AND $ila
set_property CONTROL.CAPTURE_MODE ALWAYS $ila
set_property TRIGGER_COMPARE_VALUE eq1'b1 $p
run_hw_ila $ila
```

ESPERADO: ILA armado, sin error de probes. DETENERSE si el LTX no corresponde o el dispositivo/ILA es ambiguo. Revisar BIT/LTX, clock y perfil.

SIGUIENTE: paso09 en XSDB, sin cerrar Vivado. No volver a ejecutar reset_hw_ila después de producir el evento.

10 Paso 09 / lanzar aplicación y leer estados

DÓNDE: volver a la MISMA sesión XSDB; no iniciar otra ni ejecutar reset.

ESTADO INICIAL: paso 07 correcto e ILA esperando si se hace captura.

ACCIÓN: borrar el estado anterior, cargar app ARM en el A53 detenido y ejecutarla. La app prepara/prueba DDR y libera CVA6. Se esperan 20 segundos y luego hasta 30 segundos adicionales por el estado de la ROM.

No interpretar dow completado como PASS. El criterio funcional está en el paso10.

XSDB / Tcl

```
for {set off 0} {$off < 32} {incr off 4} {
    mwr [expr {0x800004000 + $off}] 0
}
sel {Cortex-A53 #0}
dow -force $app
con
after 20000
stop
set deadline [expr {[clock milliseconds] + 30000}]
set s [rd64 0x800004000]
while {$s == 0 && [clock milliseconds] < $deadline} {
    after 500
    set s [rd64 0x800004000]
}
foreach a {0x800004000 0x800004008 0x800004010 0x800004018} {
    puts [format "0x%x = 0x%016x" $a [rd64 $a]]
}
puts [format "GPIO=0x%08x" [rd32 0xff0a0054]]
```

ESPERADO: cuatro valores de estado de 64 bits y GPIO impresos, sin errores de lectura. Guardar TODA la salida. Si el tiempo vence con estado0, registrar incompleto; no afirmar fallo de timing automáticamente.

SIGUIENTE: paso10 para evaluar esos valores, y después paso11 para guardar la captura.

11 Paso 10 / decidir PASS, FAIL o incompleto

DÓNDE: salida XSDB del paso09. No se ejecutan nuevos resets ni se repite la app durante la evaluación.

PASS funcional requiere TODOS estos valores:

0x800004000 (STATUS0): 0x0000000000000001.

0x800004008 (STATUS8): 0x0000000000000000.

0x800004010 (STATUS16): 0x0000000000000000.

0x800004018 (STATUS24): 0x0000000000000000.

GPIO en0xff0a0054: bit31 activo; por ejemplo 0x80000000. Esto indica reset liberado. No se exige que los demás bits valgan0.

FAIL de comparación: STATUS0=0xffffffffffffffff. Guardar estados secundarios y captura para identificar la diferencia; no sobrescribirlos con un nuevo intento.

INCOMPLETO: STATUS0 sigue en0 o tiene otro valor no reconocido. Investigar arranque, reset, clock y memoria. No equivale a PASS ni identifica por sí solo el origen del problema.

ERROR de herramienta: fallo de USB, targets, descarga, lectura o timeout de herramienta. Registrar ERROR, separado de un fallo funcional de la ROM.

Con ILA, el ensayo completo exige además captura con muestras. Un PASS de memoria sin CSV sólo acredita la parte funcional observada.

SIGUIENTE: paso11 incluso si hubo FAIL, para preservar la captura disponible. Después cerrar y guardar en el paso12. No repetir automáticamente.

12 Paso 11 / descargar y conservar captura

DÓNDE: volver a Tcl Console de VIVADO, donde permanece definido $ila del paso08.

ACCIÓN: esperar hasta 2 minutos por el trigger y descargar la captura a un directorio nuevo. El timeout de wait_on_hw_ila se expresa en MINUTOS.

Si no se armó ILA, no ejecutar este bloque. Si CORE_STATUS no es IDLE, detenerse y registrar que la captura no terminó.

VIVADO / Tcl Console

```
wait_on_hw_ila -timeout 2 $ila
if {[get_property CORE_STATUS $ila] ne "IDLE"} {
    error "Captura no completada"
}
set run [file join /home/nachofalchi/data/fpga-work \
    operacion cva6_ops \
    manual_[clock format [clock seconds] -format %Y%m%d_%H%M%S]]
if {[file exists $run]} {error "Salida ya existe"}
file mkdir $run
set data [upload_hw_ila_data $ila]
write_hw_ila_data -csv_file [file join $run capture.csv] $data
puts "Guardar tambien salida XSDB y archivos usados en: $run"
```

ESPERADO: archivo capture.csv en la carpeta que imprime Vivado. Abrirlo para comprobar que tiene encabezado y filas de muestras. Conservar la ruta junto a la sesión XSDB aunque se hayan creado dos carpetas distintas.

La UI muestra ondas después de subir los datos. No basta con que exista un ILA en Hardware Manager: debe haber una captura efectiva.

SIGUIENTE: paso12.

13 Paso 12 / cerrar y preparar la siguiente pasada

DÓNDE: XSDB, terminal Linux y Vivado.

1. Guardar el log de XSDB, captura CSV y una nota con fecha, perfil25 o 91, cable serial, rutas BIT/LTX/FSBL/app y los cuatro estados. Anotar PASS/FAIL/ERROR o incompleto.

2. En XSDB ejecutar disconnect y exit. Si se usó script para grabar la terminal, ejecutar exit otra vez en la subshell. Volver al prompt Linux original.

3. En Vivado conservar la captura; cerrar el hardware target propio al terminar su uso. No cerrar servidores o sesiones de otras personas.

4. Si se terminó la pasada25 con PASS, realizar una nueva pasada desde el paso03, en carpeta nueva: en el paso04 elegir 91 y en el paso08 elegir 91. Repetir TODO el arranque. El BIT de25 no se convierte en91 cambiando un registro suelto.

5. Si25 falla, investigar el entorno y consultar diagnóstico antes de 91. Si91 falla, conservar el resultado sin modificar la evidencia25.

6. Al finalizar la reunión, desconectar de las herramientas y apagar la placa. Tras cortar alimentación no se asume que el BIT o modo volátil sigan cargados.

Alcance de la conclusión: la prueba demuestra lo observado en esa sesión. No permite afirmar frecuencia máxima absoluta, validación completa del CPU ni carga de aplicaciones genéricas.

Lista final de evidencia: hashes correctos; perfil coherente; log XSDB; estados completos; captura si se hizo; rutas y fecha. No borrar ni reemplazar paquetes congelados.

XSDB y TERMINAL / cierre

```
# XSDB, al terminar:
disconnect
exit
# Si se estaba grabando con script, en la subshell Bash:
exit
```

14 A / Alternativa automatizada

Abrir una terminal Linux. Estos comandos funcionan desde cualquier directorio con el wrapper en PATH. Si “cva6” no existe, usar la ruta completa indicada debajo.

Con la placa encendida y conectada, repetir primero 25 MHz. Si falla, resolver el entorno antes de intentar 90,9 MHz.

TERMINAL / Bash

```
cva6 estado
cva6 verificar
cva6 ensayo --frecuencia 25 --ejecutar
cva6 ensayo --frecuencia 91 --con-ila --ejecutar

# Si el comando no aparece en PATH:
/home/nachofalchi/.local/bin/cva6 verificar
/home/nachofalchi/.local/bin/cva6 ensayo \
  --frecuencia 91 --con-ila --ejecutar

# Plan sin tocar hardware:
cva6 ensayo --frecuencia 91 --con-ila
```

ensayo coordina FSBL, BIT, reset, aplicación y captura. Sin --ejecutar sólo planifica. verificar comprueba archivos; no valida la placa.

Con ILA, PASS requiere aplicación PASS y captura con muestras. READY sólo significa preparado; CAPTURED sólo significa captura obtenida.

La salida indica una carpeta nueva bajo /home/nachofalchi/data/fpga-work/operacion/cva6_ops/. Conservar result.json, console.log, hardware_status.tsv, plan.json, manifest.json y capture.csv de sus etapas.

Si hay errores, detenerse e inspeccionar los logs. No borrar locks para forzar ejecuciones. La vía manual siguiente es independiente del wrapper, pero usa los mismos archivos.

15 B / Consolas y herramientas

Ventana A - terminal Linux: comandos como cva6, lsusb, sha256sum y arranque de herramientas. Prompt usual terminado en $. Las variables se definen NOMBRE=valor.

Ventana B - XSDB: se inicia desde la terminal. Su prompt es xsdb%. Aquí van connect, targets, rst, dow, con, stop, mrd, mwr y fpga. Es Tcl: variables con set nombre valor.

Ventana C - Vivado: menú Hardware Manager e ILA. Su pestaña Tcl Console usa Tcl, pero las órdenes son de Vivado: get_hw_devices, get_hw_ilas, run_hw_ila. No pegar estas órdenes en XSDB.

Ventana D - servidor opcional: hw_server, sólo si no había uno local escuchando. Mantenerlo abierto. Si Hardware Manager ya lo inició, reutilizarlo.

Abrir Vivado no requiere abrir un proyecto ni un DCP para acceder al hardware. Si ya está abierto, reutilizar la ventana. El menú Flow está arriba entre Edit y Tools.

No correr cva6 ensayo mientras se está realizando el procedimiento manual. Los locks de la automatización no pueden impedir acciones manuales desde otra herramienta.

En Bash se usa barra invertida al final de una línea para continuar el comando. Copiar el bloque entero. En XSDB los bloques con llaves deben copiarse completos; si aparece prompt de continuación, terminar el bloque, no pegar comandos de otro apartado.

TERMINAL / Bash

```
# Ventana A: iniciar Vivado sólo si no está abierto.
/home/nachofalchi/data/fpga-work/Xilinx/2026.1/\
Vivado/bin/vivado

# Otra terminal: iniciar XSDB.
/home/nachofalchi/data/fpga-work/Xilinx/2026.1/\
Vitis/bin/xsdb
```

16 C / Programar el BIT desde la UI

Esta alternativa describe el perfil91. Para25, seleccionar BIT y LTX del paquete conocido25. En ambos casos sustituye sólo la programación del paso06; el resto de la secuencia se conserva.

Si se prefiere el botón de Vivado, sustituir exclusivamente las líneas “sel PSU; fpga -f $bit; after 3000” del paso06 por estos pasos. Mantener el resto de la secuencia XSDB intacta.

1. Ejecutar en XSDB hasta finalizar la PRIMERA ejecución de FSBL (stop). No ejecutar todavía la segunda.

2. En Hardware Manager seleccionar el dispositivo de la misma placa; clic derecho > Program Device.

3. En Bitstream file seleccionar kr260_top.bit dentro del paquete timing_closed_91mhz_2026-10-22. Verificar toda la ruta: el nombre por sí solo no distingue experimentos.

4. En Debug probes file seleccionar kr260_top.ltx de ESA misma carpeta. No seleccionar un BIT de impl_1 ni del paquete25.

5. Pulsar Program y esperar que termine sin errores.

6. Volver a XSDB y continuar con la SEGUNDA ejecución de FSBL, reset final de A53 y comprobaciones de preparación.

7. Recién después refrescar los probes en Vivado, armar ILA y lanzar la aplicación con XSDB.

No pulsar Program Device después de preparar/armar ILA. Si se hizo, volver a empezar la preparación completa. No sustituir FSBL91 por un archivo de25 aunque Vivado acepte el BIT.

Esta alternativa usa exactamente los mismos artefactos, pero introduce acciones manuales adicionales. Para una reproducción fiel y menos ambigua, el método principal del manual programa con XSDB.

17 D / Diagnóstico y recuperación

Falla verificar por hashes: no tocar hardware. Identificar el archivo que difiere y compararlo con el manifiesto congelado. Sólo la autoentrada histórica del manifiesto25 es una excepción documentada; ninguna diferencia de ELF/BIT/LTX se ignora.

Falta cva6: usar su ruta completa; si el wrapper falta también, los apartados XSDB/Vivado son independientes. No se necesita instalar otro compilador para ejecutar los ELF preparados.

Falta hw_server o puerto ocupado: pgrep -af hw_server permite ver procesos. Si ya hay uno local, reutilizarlo; si no, iniciarlo con la ruta del manual. No ejecutar killall ni borrar locks de la suite.

XSDB no encuentra Cortex-A53 #0: ejecutar targets y guardar la lista. Revisar PSU, cable y alimentación. El selector del manual exige un target exacto; no cambiar a otro procesador por ensayo y error.

Clock incorrecto: comprobar el FSBL elegido y si se ejecutó después del BIT. No escribir un divisor suelto para esconder el fallo. Repetir secuencia coherente.

Reset no retenido: no cargar app ni armar captura como si todo estuviera listo. Confirmar FSBL parcheado y registros GPIO. El VIO no debe cambiarse arbitrariamente.

STATUS0=0: puede ser que CVA6 no arrancó o no terminó. Revisar reset, clock, DDR, app y errores anteriores; no declarar fallo de timing automáticamente.

STATUS0=-1: conservar estados secundarios y captura; corresponde a un reporte de comparación fallida de la ROM de test.

PASS de memoria pero sin CSV: hay evidencia funcional parcial, pero no un ensayo completo con ILA. Repetir con preparación nueva y captura armada antes del release.

USB desconectado, logout o herramienta terminada: guardar lo que quedó; no adivinar el estado. La ruta segura de reproducción es preparar nuevamente, usando carpetas nuevas. Los comandos de hardware no se reintentan solos.

18 E / Consultar timing en Vivado

Para analizar timing, no hace falta conectar la placa. En Vivado: File > Checkpoint > Open; abrir kr260_top_91mhz_closed.dcp del paquete91. Luego Reports > Timing > Report Timing Summary > OK. Si aparece una opción de nueva ventana, dejarla desmarcada.

Para probar hardware no hace falta abrir el DCP ni el proyecto. Hardware Manager programa el BIT; la inicialización del PS/DDR se realiza con XSDB. El resultado de impl_1 no es el BIT cerrado seleccionado.

Verificación independiente del wrapper (Bash):

TERMINAL / Bash

```
CVA6_BASE=/home/nachofalchi/data/fpga-work
CVA6_BOOT="$CVA6_BASE/cva6-equipo/corev_apu/fpga/software"
CVA6_PKG="$CVA6_BOOT/kr260_ps_boot/timing_closed_91mhz_2026-10-22"
(cd "$CVA6_PKG" && sha256sum -c SHA256SUMS.txt)
sha256sum "$CVA6_BASE/operacion/cva6_ops/fsbl91/\
build_20261029_1831/kr260_fsbl_91mhz_patched.elf"
```

SHA256 esperado del FSBL91 (unir las dos mitades):

4b2c98d7ea93099d62b7538ce4d92b071

d3672ef157b48acf7bf688df380ab87

El paquete91 debe verificar 29 archivos. El manifiesto histórico25 tiene una autoentrada incorrecta conocida; esto no autoriza ignorar fallos de los binarios.

Fuente operativa: automatizacion/cva6_ops/templates_board/board_run.tcl e ila_capture.tcl; board_ops.py, config.json y README.md. La secuencia de este manual se contrastó con esos archivos el 30/10/2026. No se ejecutó hardware al elaborar el manual.

Referencia AMD: docs.amd.com/r/en-US/conversion-methodology/Timing-Closure

Referencia ILA: docs.amd.com/r/2021.2-English/ug835-vivado-tcl-commands/run_hw_ila

El procedimiento manual no incluye los locks, la validación de identidad ni los registros automáticos completos de ensayo. Preferir la vía automatizada cuando funcione. Este manual no promete PASS ni modifica el CVA6.

19 F / Profesor: resultados

¿Qué se logró? Una implementación que cumple STA a90,908180 MHz y un paquete reproducible para probarla. El conocido bueno25 tiene PASS funcional histórico. La validación hardware91 debe obtenerse y registrarse; no se puede dar por realizada.

¿100 MHz cerró? No. El mejor resultado100 adoptado en la documentación tiene WNS -0,553ns, TNS -322,607ns y1285 endpoints negativos. Consultar CSV de evaluaciones para el historial completo; no afirmar que se probaron todas las combinaciones posibles.

¿Por qué bajar a90,9? Hay un resultado completo con clock configurado por IOPLL/11 y STA cerrado. Bajar frecuencia amplía el tiempo disponible por ciclo. No demuestra que90,9 sea el máximo absoluto de cualquier implementación.

¿Hay margen? Setup+0,001ns y hold+0,004ns: cumple el análisis guardado, con margen muy pequeño. No venderlo como cierre holgado ni robustez funcional demostrada.

¿La placa ya corrió91? No según el estado documental al preparar este manual. FSBL compilado y bitstream existente no son una ejecución hardware.

¿El clock es91 exacto? No:90,908180 MHz. El análisis usa período11ns del clock generado por el IP del PS. La lectura de divisor confirma configuración, no medición con instrumental.

¿Podemos garantizar100 mañana? No hay evidencia para garantizarlo. Un bitstream generado tampoco prueba por sí solo cierre de timing.

¿Está medido consumo, rendimiento o IPC? No con esta batería de cierre/bring-up. No presentar frecuencia como un benchmark de rendimiento.

Evidencia para responder: README del paquete91, reports/timing_summary.rpt, docs/seguimiento_fpga/evaluaciones_100mhz.csv y resultados_clave.csv.

20 G / Profesor: arquitectura y prueba

¿Qué hace PS y qué hace PL? El ARM del PS ejecuta FSBL y la app de preparación/prueba. El CVA6 está implementado en PL y accede a la DDR del PS a través del camino AXI/HP0 preparado.

¿El FSBL es el programa de CVA6? No: es ELF AArch64 del ARM. Inicializa PS/DDR/clock y retiene reset. La aplicación ARM también es distinta del código RISC-V de la BootROM de test.

¿Qué comprueba PASS? El resultado que escribe la ROM de test en memoria, estados secundarios sin diferencias y reset liberado; con ILA se exige además una captura. Es bring-up de ROM/DDR, no un test exhaustivo de ISA, Linux, caches o todos los periféricos.

¿Por qué escribir DDR desde ARM? Es parte del test existente y su preparación. Puede destruir programas precargados. No es el mecanismo definitivo para cargar aplicaciones propias.

¿Direcciones? La ventana CVA6 0x80000000-0xBFFFFFFF se traduce a PS 0x800000000-0x83FFFFFF. El estado visto por CVA6 en0x80004000 aparece desde PS en0x800004000. Cuidado con el dígito extra.

¿UART? Esta demo no depende de UART: se leen estados por JTAG. No esperar mensajes serie como criterio de éxito.

¿ILA prueba todo? No. Observa una ventana finita y señales concretas; ausencia de lecturas en una ventana no prueba ausencia global. El trigger por commit no captura necesariamente todos los accesos DDR.

¿Se modificó CPU para este manual? No. No se cambian RTL, parámetros ni microarquitectura por preparar/ejecutar la guía. El proyecto ya contiene modificaciones de integración previas del equipo; no afirmar que todo el repositorio es idéntico al upstream.

¿Restaurar ROM original y cargar DDR ya está hecho? No. Es un plan documental separado. No usar instrucciones antiguas sobre macros como evidencia de selección actual: consultar la auditoría y el plan actualizados.

21 H / Profesor: timing y evidencia

¿WNS? El slack peor de setup. Negativo implica al menos una restricción incumplida; positivo indica margen en el peor camino analizado. ¿TNS? Suma de slacks negativos; cero es el objetivo, junto a ausencia de endpoints fallando.

¿Hold? Restricción de llegada demasiado temprana. WHS es el peor slack de hold y THS acumula violaciones. Cerrar hold no compensa fallar setup.

¿Síntesis cerrada garantiza ruteo? No. El timing de síntesis usa estimaciones; colocación, ruteo y optimizaciones físicas determinan los retardos finales. Comparar siempre el post-route del experimento correcto.

¿75 endpoints JTAG? El paquete91 documenta75 endpoints sin restricción asociados al tck de JTAG atado a0. Hay que mostrar check_timing.rpt y el contexto de integración; no decir “todo está restringido” ni confundirlos con endpoints funcionales de setup fallando.

¿Qué receta produjo91? Síntesis Vivado Synthesis Defaults; implementación Performance_ExploreWithRemap, route NoTimingRelaxation con tns_cleanup y phys_opt post-route Explore; una pasada AggressiveExplore sobre el ruteado llevó el resultado de WNS -0,065ns a+0,001ns. Ver scripts y logs del paquete para los comandos reales.

¿Cambiar directivas cambia el CPU? Cambia la realización netlist/física buscando preservar la función HDL; no se está editando la microarquitectura. Eso no sustituye pruebas de equivalencia o validación funcional completa.

¿El .xpr reproduce automáticamente el mejor resultado? No. El checkpoint y BIT congelados identifican el resultado final; impl_1 puede ser un estado previo. No regenerar encima durante la reunión.

¿Cómo se liga evidencia a binarios? Con SHA256, manifiestos, configuración del FSBL y carpetas por sesión. Conservar logs y CSV con los archivos exactos usados. Un nombre idéntico no prueba identidad.

¿Fmax=1/(11ns-1ps)? No usar esa cuenta como máximo certificado. El máximo depende de restricciones, incertidumbre, implementación y validación; sólo afirmar el punto analizado que efectivamente pasó.

22 I / Rutas: proyecto y placa

Base real de todo el trabajo:

/home/nachofalchi/data/fpga-work

Las rutas que siguen son RELATIVAS a esa base; concatenarlas para obtener la ruta absoluta. No son accesos directos.

Proyecto y fuentes: cva6-equipo/

Proyecto Vivado: cva6-equipo/corev_apu/fpga/ariane_kr260_debug/ariane_kr260_debug.xpr

Software de arranque: cva6-equipo/corev_apu/fpga/software/kr260_ps_boot/

Paquete91: dentro del software de arranque, timing_closed_91mhz_2026-10-22/

BIT: paquete91/kr260_top.bit. Probes: paquete91/kr260_top.ltx.

Checkpoint cerrado: paquete91/kr260_top_91mhz_closed.dcp.

Hardware exportado: paquete91/kr260_cva6_91mhz.xsa.

FSBL91: operacion/cva6_ops/fsbl91/build_20261029_1831/kr260_fsbl_91mhz_patched.elf.

Identidad del build: en esa misma carpeta fsbl91_manifest.json y result.json.

Referencia25: software de arranque/known_good_2026-09-18/.

App ARM: referencia25/kr260_app_jtag.elf. FSBL25: referencia25/kr260_fsbl_patched.elf.

Automatización: automatizacion/README.md; config.json; cva6_ops/board_ops.py; cva6_ops/session_ops.py; cva6_ops/templates_board/.

Resultados nuevos: operacion/cva6_ops/. La salida de cada comando informa su subcarpeta.

No mover ni borrar paquetes congelados. Para abrir las carpetas desde Bash, usar las órdenes de la página final. Para evitar ambigüedad, el índice de rutas también se entrega como texto.

23 J / Rutas: reportes y documentación

Dentro de paquete91/reports/:

timing_summary.rpt: métricas globales de setup/hold.

critical_setup.rpt y critical_hold.rpt: detalle de caminos críticos.

clocks.rpt: clocks reconocidos. pulse_width.rpt: ancho de pulso.

check_timing.rpt: advertencias de cobertura/restricciones.

route_status.rpt: estado de ruteo. drc.rpt: reglas de diseño.

methodology.rpt y bus_skew.rpt: verificaciones complementarias.

Dentro de paquete91/scripts/: Tcl y logs de creación del resultado cerrado.

Dentro de paquete91/project_state/: instantánea de XPR, BD, XCI y diff del proyecto de ese momento.

Documentación unificada: cva6-equipo/docs/seguimiento_fpga/README.md.

Estado y resultados: mismo directorio/estado_y_resultados.md.

Datos de campañas: evaluaciones_100mhz.csv; resultados_clave.csv; historial_pruebas.md; metodologia_y_decisiones.md.

BootROM/DDR futuro: plan_bootrom_original_y_programas_ddr.md. Auditoría: auditorias/bootrom_y_carga_ddr_20261029.json.

Explicación del camino de memoria: cva6-equipo/docs/kr260_ps_ddr_architecture.md y kr260_ddr_integration.md. Algunos documentos antiguos describen estados previos; usar el plan/auditoría actual para la selección real de ROM.

Entrega reunión: entregas/reunion_profesor_20261030/.

Resumen PDF: esa entrega/output/pdf/CVA6_KR260_estado_reunion_20261030.pdf.

Manual actual: esa entrega/manual_placa/output/pdf/Manual_CVA6_KR260_90_9MHz.pdf.

Respaldo externo registrado: RESPALDO_EXTERNO_20261029.md. El destino conocido es nacho@192.168.86.31 bajo /home/nacho/backups/CVA6. Los respaldos son copias, no sustituyen archivos locales. No se garantiza acceso fuera de esa red.

24 K / Acceso rápido y referencias

Abrir directorios reales desde la terminal Linux. Estas órdenes no programan hardware. El archivo RUTAS_IMPORTANTES.txt contiene las rutas absolutas comprobadas.

TERMINAL / Bash

```
CVA6_BASE=/home/nachofalchi/data/fpga-work
cd "$CVA6_BASE/cva6-equipo"
# Abrir carpeta del proyecto:
xdg-open "$CVA6_BASE/cva6-equipo/corev_apu/fpga/ariane_kr260_debug"
# Abrir documentación y entrega:
xdg-open "$CVA6_BASE/cva6-equipo/docs/seguimiento_fpga"
xdg-open "$CVA6_BASE/entregas/reunion_profesor_20261030"
# Ver comandos disponibles sin ejecutar pruebas:
/home/nachofalchi/.local/bin/cva6 --help
/home/nachofalchi/.local/bin/cva6 ensayo --help
```

Fuentes operativas locales: plantillas board_run.tcl e ila_capture.tcl y configuración de automatización. Se contrastó el orden sin lanzar herramientas de hardware.

Conexiones J4/J12 - AMD (se usan sólo sus datos de conectores; la demo no sigue su arranque Linux):

https://xilinx.github.io/kria-apps-docs/kr260/linux_boot/embedded_linux/build/html/docs/connections.html

SW1/FWUEN - AMD UG1092:

https://docs.amd.com/r/en-US/ug1092-kr260-starter-kit/Firmware-Update

Checkpoint/timing - AMD:

https://docs.amd.com/r/en-US/conversion-methodology/Timing-Closure

ILA - AMD UG835:

https://docs.amd.com/r/2021.2-English/ug835-vivado-tcl-commands/run_hw_ila

Respuestas preparadas para preguntas probables; no son garantía de cubrir cualquier pregunta. Si falta evidencia, responder que no se midió o que sigue pendiente, y mostrar el archivo fuente relevante.